

최적화 그랜드 챌린지 2026 (OGC 2026)

그랜드 조선소 퍼즐: 블록을 배치하고, 시간을 이겨라

OGC 2026 조직위원회

2026년 6월 15일 (문서 버전 1.2)

초록

최적화 그랜드 챌린지(OGC) 2026은 조선업에서 도출된 공간적 블록 스케줄링 문제를 다룹니다. 조선소에서는 선박의 각 구조적 블록이 베이(bay)라고 불리는 고정된 작업 공간 안에서 제작됩니다. 블록은 불규칙하고 층으로 이루어진 형태를 가진 3차원 물체이므로, 겹침 없이 베이에 배치하는 것 자체가 어려운 기하학적 문제입니다. 여기에 더해 각 블록마다 반입 가능 시점(release date), 처리 시간(processing time), 납기일(due date)이 있으며, 베이 공간은 공유되기 때문에 주변 블록들로 인해 블록이 대기하거나 필요 이상으로 오래 머물러야 할 수 있습니다. 따라서 알고리즘은 각 블록에 대해 어느 베이에 배치할지, 어디에 어떤 방향으로 놓을지, 언제 반입하고 반출할지를 모두 동시에 결정해야 합니다. 이 문서는 공식 문제 정의, 참가 안내, 그리고 평가 기준을 제공합니다.

문서 업데이트: 이 문서는 오류 수정이나 모호한 내용 명확화를 위해 개정될 수 있습니다. 최신 버전은 대회 웹사이트에서 항상 확인할 수 있으며, 표지의 버전 번호가 현재 읽고 있는 개정판을 나타냅니다. 참가자는 웹사이트에서 업데이트를 직접 확인할 책임이 있으며, 중요한 변경 사항은 웹사이트에서도 공지됩니다.

1. 문제 설명

1.1 용어 정의

- 베이(Bay): 조선소 내에서 작업이 수행되는 물리적으로 구획된 작업 공간입니다. 각 베이 $j \in M$ 은 고정된 크기 $W_j \times H_j$ 를 가지며, $M := \{1, 2, \dots, m\}$ 은 베이들의 집합입니다.

- 블록(Block): 선박의 대형 구조 부품을 나타내는 생산 단위입니다. 각 블록 $i \in N$ 은 3D 구조를 형성하는 K_i 개의 다각형 층(layer)으로 구성되며, $N := \{1, 2, \dots, n\}$ 은 블록들의 집합입니다. 블록은 회전할 수 있으며, 가능한 방향 옵션의 수는 O_i 로 표시됩니다.
- 반입 가능 시점 (R_i): 블록 i 가 처리를 시작할 수 있는 가장 이른 시점입니다.
- 납기일 (D_i): 블록 i 의 희망 완료 시점입니다.
- 처리 시간 (P_i): 블록 i 를 처리하는 데 필요한 시간입니다.
- 반입 시점 ($ENTRY_i$): 블록 i 가 베이에 배치되는 시점입니다.
- 반출 시점 ($EXIT_i$): 블록 i 가 베이에서 나가는 시점입니다.
- 지연 (T_i): 납기일을 초과한 지연 시간으로, 다음과 같이 정의됩니다:

$$T_i = \max(0, EXIT_i - D_i)$$
- 작업량 (L_i): 블록 i 처리 중에 필요한 총 작업량입니다.
- 선호도 (S_{ij}): 블록 i 가 베이 j 에 배정될 때의 선호도 점수입니다.

1.2 문제 설정

조선소에서 블록은 여러 생산 단계를 거치며, 그 중 일부는 베이 내에서 수행됩니다. 각 베이는 크레인, 인력, 운송 경로 등의 자원이 공유되는 고정 작업 공간입니다. 블록은 베이에 배정되어 처리되어야 하며, 이 과정에서 시간적 제약과 공간적 제약을 모두 만족해야 합니다. 각 베이는 공간적으로 겹치지 않는 한 여러 블록을 동시에 수용할 수 있습니다. 각 블록은 고유한 반입 가능 시점, 처리 시간, 납기일을 가집니다. 따라서 배정 및 스케줄링 결정은 공간적 실현 가능성과 시간적 실현 가능성을 동시에 고려해야 합니다.

1.3 베이

각 베이는 독립적인 작업 공간으로, 베이 간에는 물리적 또는 운영상의 분리가 존재합니다. 조선소에는 일반적으로 여러 베이가 있습니다. 우리는 조선소가 m 개의 베이로 구성되며, 각 베이 j 의 크기가 $W_j \times H_j$ 라고 가정합니다. 주어진 블록에 대해 다음 결정들이 동시에 이루어져야 합니다:

1. 어느 베이에 배정할지
2. 베이 내에서의 위치와 방향
3. 블록을 베이에 반입하는 시점

4. 블록을 베이에서 반출하는 시점

각 베이에는 독립적인 크레인이 있으므로, 베이 내의 모든 작업은 독립적으로 수행될 수 있습니다. 이번 대회에서 다루는 작업은 ENTRY와 EXIT 두 가지로, 각각 블록을 베이에 반입하고 반출하는 것을 의미합니다. 하루에 수행할 수 있는 작업의 수에는 제한이 없으므로, 그날의 작업 시작 전에 임의의 수의 작업을 수행할 수 있습니다. 예를 들어, 처리 시간이 3인 블록이 5일에 베이에 반입(ENTRY 작업)된다면, 가장 빠른 반출(EXIT 작업) 가능일은 8일입니다. 즉, 해당 블록은 5일부터 7일까지 처리될 수 있습니다. 단, 작업 순서도 중요한데, 각 작업이 베이의 상태를 변경하기 때문입니다. 모든 EXIT 작업은 ENTRY 작업보다 먼저 수행되어야 하며, 같은 종류의 작업들 중에서는 알고리즘이 수행 순서를 결정해야 합니다.

1.4 블록

총 n 개의 블록이 주어집니다. 각 블록은 3D 구조를 형성하는 여러 다각형 층으로 구성한다고 가정합니다. 각 층을 정의하는 다각형은 볼록(convex)일 필요가 없습니다. 각 블록 i 는 K_i 개의 층으로 구성되며, 모든 층의 높이는 동일하다고 가정합니다. '빈' 층은 존재할 수 없으며, 인접한 두 층은 물리적으로 연결되어 있으므로 '떠 있는' 층은 불가능합니다.

블록을 배치할 때는 방향(orientation)도 함께 결정해야 합니다. 이번 대회에서는 각 블록에 대해 가능한 방향 옵션의 수가 미리 정해진 문제 인스턴스를 제공하며, 이는 블록 i 에 대해 O_i 로 표시됩니다.

각 블록 i 는 작업량 L_i 와 각 베이에 대한 선호도 점수 S_{ij} 도 가지고 있습니다. 작업량은 해당 블록에 대해 수행해야 할 작업의 양을 나타내며, 베이 간 총 작업량의 분산이 작을수록 좋은 해입니다. 선호도 점수 S_{ij} 는 블록 i 가 베이 j 에 배정되는 것을 얼마나 선호하는지를 나타냅니다.

1.5 결정 변수

각 블록 i 에 대해 다음을 결정해야 합니다:

- 베이 j 로의 배정
- 베이 내 배치 위치
- 방향 선택
- 반입 시점 $ENTRY_i$ 및 반출 시점 $EXIT_i$

1.6 제약 조건

제약 조건은 크게 세 가지 그룹으로 나뉩니다. 첫 번째는 모든 블록을 베이에 배정하고 ENTRY/EXIT 작업을 보장하는 것입니다. 두 번째는 블록이 최소 필요 처리 시간 이상 머문 후에야 작업이 완료되도록 하는 시간적 제약입니다. 세 번째는 베이 내 블록들의 공간적 실현 가능성에 관한 제약입니다. 크레인 작업은 하루 업무 시작 전에 수행될 수 있으므로, $ENTRY_i$ 와 $EXIT_i$ 를 갖는 블록 i 는 배정된 베이를 $[ENTRY_i, EXIT_i)$ 기간 동안 점유합니다.

1.6.1 배정 제약

배정 제약: 모든 블록은 반드시 하나의 베이에 배정되어야 합니다. 배정되지 않은 블록이 존재해서는 안 됩니다.

작업 제약: 모든 블록은 정확히 하나의 ENTRY 작업과 하나의 EXIT 작업을 가져야 합니다. 블록은 베이에 배치된 후 이동하거나 회전할 수 없습니다.

1.6.2 시간적 제약

반입 가능 시점 제약: 블록은 반입 가능 시점 이전에 처리를 시작할 수 없습니다:

$$ENTRY_i \geq R_i$$

처리 시간 제약: 블록의 작업은 반출 전에 완료되어야 합니다:

$$EXIT_i - ENTRY_i \geq P_i$$

블록은 필요한 처리 시간이 지난 후에도 베이에 '남아 있을' 수 있습니다. 이는 다른 블록들로 인해 반출이 막히는 경우 발생합니다. 반출 시점 $EXIT_i$ 가 납기일 D_i 를 초과하면 양의 지연이 발생합니다. 일반적으로 블록은 $D_i - R_i > P_i$ 이므로, 베이의 다른 블록들을 고려하여 최적의 반입/반출 시점을 결정할 '여유(slack)' 시간이 존재합니다.

1.6.3 공간적 제약

$P_{o,x,y}(i,l)$ 을 위치 (x, y) 에 방향 o 로 놓인 블록 i 의 층 l 의 다각형이라 할 때, 같은 베이의 같은 날에 있는 두 블록 i_1 과 i_2 사이에 충돌이 발생한다는 것은 $\text{int}(P_{o_1,x_1,y_1}(i_1,l)) \cap \text{int}(P_{o_2,x_2,y_2}(i_2,l)) \neq \emptyset$ 인 경우를 의미합니다. 여기서 $\text{int}(\cdot)$ 는 (닫힌) 다각형의 내부 점들의 집합을 나타냅니다. 즉, 두 블록은 모든 층 l 에 대해 내부 점 교차가 없을 때 충돌 없음 (collision-free) 상태입니다. 두 블록이 다각형의 모서리를 공유하는 것은 허용됩니다.

베이 포함 제약: 각 블록은 배정된 베이 내에 완전히 포함되어야 합니다.

충돌 방지 제약: 날짜 t , 베이 j , $N(t,j)$ 의 서로 다른 두 블록 i_1, i_2 에 대해, 결정 변수 $(o_1, x_1, y_1, o_2, x_2, y_2)$ 는 모든 층 $l = 1, 2, \dots, \min\{K_{i_1}, K_{i_2}\}$ 에 대해 $C(i_1, l, o_1, x_1, y_1, i_2, l, o_2, x_2, y_2) = 0$ 을 만족해야 합니다. 쉽게 말해, 같은 날 같은 베이에 있는 두 블록은 같은 레벨의 층이 충돌하지 않을 때 공간적으로 실현 가능합니다.

크레인 작업 제약: 블록을 반입하거나 반출하기 위해서는 크레인이 블록을 수직으로 간섭 없이 이동할 수 있어야 합니다. 날짜 t 와 베이 j 에서 블록 i_1 의 크레인 작업(ENTRY 또는 EXIT)은, $i_2 \in N(t,j) \setminus \{i_1\}$ 인 모든 블록에 대해 $l_1 \leq l_2$ 를 만족하는 모든 l_1, l_2 에 대해 $C(i_1, l_1, o_1, x_1, y_1, i_2, l_2, o_2, x_2, y_2) = 0$ 일 때만 실현 가능합니다. 즉, 블록을 반입/반출하려면 해당 블록의 각 층이 같거나 높은 레벨에 있는 다른 블록들의 층과 공간적으로 충돌하지 않아야 합니다.

1.7 목적 함수

목적은 세 가지 기준의 가중 합을 최소화하는 것입니다:

총 지연 (Z1): 모든 블록의 총 지연 합계입니다.

$$Z1 := \sum_{i=1}^n T_i$$

작업량 불균형 (Z2): 베이 간 최대 정규화 작업량 불균형입니다.

$$Z2 := \lceil \max_{j_1 \neq j_2} |u_{j_1} \cdot \sum_{i \in N(j_1)} L_i - u_{j_2} \cdot \sum_{i \in N(j_2)} L_i| \rceil$$

여기서 $N(j) \subseteq N$ 은 베이 j 에 배정된 블록들의 집합이고, u_j 는 블록이 배치된 베이에 부여된 가중치입니다. 이 값은 모든 베이의 평균 면적을 해당 베이의 면적으로 나눈 값으로 정의됩니다:

$$u_j = (\sum_{k \in M} W_k \times H_k / m) / (W_j \times H_j)$$

더 큰 베이는 더 작은 가중치 값을 가지며, 이는 큰 베이에 블록을 배치하면 같은 작업량이라도 혼잡도가 낮아진다는 것을 반영합니다.

총 선호도 점수 (Z3): 배정된 베이에 대한 선호도 점수의 총합입니다.

$$Z3 := \sum_{j \in M} \sum_{i \in N(j)} (S_{\max_i} - S_{ij}), \text{ 여기서 } S_{\max_i} := \max_{j \in M} \{S_{ij}\}$$

모든 블록이 가장 선호하는 베이에 배정되면 $Z3 = 0$ 이 됩니다.

최종적으로 최소화할 목적 함수는 다음과 같습니다:

$$w1 \cdot Z1 + w2 \cdot Z2 + w3 \cdot Z3$$

여기서 $w1$, $w2$, $w3$ 은 주어진 음수가 아닌 가중치 매개변수입니다.

2. 문제 인스턴스 정의

참가자들에게는 OGC 위원회가 생성한 '훈련용' 문제 인스턴스가 제공됩니다. 이 인스턴스들은 실제 조선 문제를 최대한 유사하게 모방하도록 설계되었으며, 다양한 어려운 측면을 포함합니다. 예를 들어, 블록이 많은 인스턴스도 있고, 블록 반입 가능/납기 일정이 더 촘촘한 인스턴스도 있습니다. 참가자들은 다양한 상황에서 잘 작동하는 알고리즘을 개발하기 위해 이러한 다양한 도전 요소들을 고려하여 알고리즘을 개발하길 기대합니다.

단일 문제 인스턴스는 다음과 같은 json 형식으로 제공됩니다:

```
{ "name": "training_problem_01",
  "bays": [
    { "width": 125, "height": 15 },
    { "width": 71, "height": 18 },
    { "width": 140, "height": 25 }
  ],
  "blocks": [
    {
      "release_time": 37,
      "due_date": 44,
      "processing_time": 7,
      "workload": 20,
      "bay_preferences": [19, 26, 55],
      "shape": [ { "orientation": 0, "layers": [ [...], [...] ] }, ... ]
    },
    ...
  ],
  "weights": { "w1": 26667, "w2": 10, "w3": 300 }
}
```

문제 인스턴스 파일은 네 부분으로 구성됩니다: (1) 문제 이름("name"), (2) 베이 목록("bays"), (3) 블록 목록("blocks"), (4) 목적 함수 가중치("weights").

베이는 0부터 $m-1$ 까지 인덱싱됩니다. 예를 들어, 베이 0의 크기는 125×15 입니다. 블록도 마찬가지로 0부터 $n-1$ 까지 인덱싱됩니다. 예를 들어, 블록 0의 반입 가능 시점은 37, 납기일

은 44이며, 베이 선호도는 19, 26, 55입니다(즉 $S00=19$, $S01=26$, $S02=55$). 단일 블록의 선호도 점수 합계는 항상 100입니다. 모든 시간 관련 값과 선호도 점수는 항상 정수입니다.

블록의 "shape"는 각 방향에 대한 층 다각형을 기술합니다. 층 다각형 목록은 가장 낮은 층부터 높은 층 순서로 꼭짓점 쌍을 포함합니다. 첫 번째 층(layer 0)의 첫 번째 꼭짓점은 항상 $[0.0, 0.0]$ 이며, 이를 기준점(reference point)이라 합니다. 동일한 방향 형태의 다른 모든 꼭짓점들은 기준점에 대한 상대 좌표입니다. 알고리즘이 블록의 위치를 (x, y) 로 결정하면, 베이 내 블록 형태의 꼭짓점들은 베이 원점에서 (x, y) 만큼 이동됩니다.

층의 꼭짓점은 기준점을 제외하고 소수점 이하 최대 4자리의 소수 좌표를 가질 수 있습니다. 블록 형태가 매우 불규칙할 수 있으므로, 블록 간 공간 충돌 검사 시 수치적 불안정성이 발생할 수 있습니다. 평가 서버도 제공된 `utils.py`의 함수를 사용하여 결과 솔루션의 실현 가능성을 검사하므로, 알고리즘의 최종 단계에서 적어도 이 함수들을 사용하는 것을 강력히 권장합니다. 모든 인덱스(베이 및 블록 포함)는 문제 인스턴스와 알고리즘 솔루션에서 0부터 시작합니다.

3. 알고리즘 제출 및 평가

팀들은 알고리즘 코드를 `submission@optichallenge.com`으로 이메일을 통해 평가 서버에 제출해야 합니다. 알고리즘은 전용 서버의 '비공개' 문제 인스턴스에서 평가되며, 결과는 다른 팀의 결과와 순위를 비교하여 리더보드를 구성하고 대회 웹페이지에 게시됩니다.

3.1 알고리즘 제출

평가 서버에 알고리즘 코드를 제출하기 위해서는 다음 요건을 충족해야 합니다:

- 이메일은 `submission@optichallenge.com` 으로 발송해야 합니다.
- 이메일은 등록 시 사용한 이메일 주소로 발송해야 합니다.
- 이메일에는 하나의 zip 파일이 첨부되어야 합니다.
- zip 파일에는 `myalgorithm.py` 가 포함되어야 합니다.
- `utils.py` 파일이 zip 에 포함된 경우 수정할 수 없습니다(알고리즘 실행 전 덮어씌워집니다).
- `myalgorithm.py` 파일은 압축 해제된 파일의 루트에 있어야 합니다(하위 폴더 내 불가).
- zip 파일 크기는 15MB 를 초과할 수 없습니다.

- 이전 수락된 제출 시간이 사전 정해진 대기 기간(12 시간) 이전이 아니면 새 제출이 수락되지 않습니다.

위 규정 중 하나라도 만족하지 않는 제출은 거부됩니다. 서버는 제출된 알고리즘 코드가 유효한지 확인하기 위해 추가적인 기본 검사(예: 손상된 zip 파일 확인)를 수행합니다. 제출이 수락되면 알고리즘 코드가 실행되어 비공개 인스턴스를 풀게 됩니다. 모든 평가는 가장 최근에 수락된 제출을 기준으로 합니다. 단, 수락이 알고리즘의 정상 실행을 보장하지는 않습니다. 누락된 의존성 등으로 인해 충돌하거나 실행에 실패할 수도 있습니다. 성공 여부와 관계없이, 가장 최근에 수락된 제출이 리더보드에 사용됩니다. 대기 타이머는 제출이 수락된 후에만 시작됩니다.

3.2 알고리즘 평가 서버

팀들은 훈련 문제를 사용하여 알고리즘을 개발해야 합니다. 제출된 알고리즘 코드는 평가를 위해 비공개 문제에서 실행됩니다. 평가 서버 사양은 다음과 같습니다:

- AMD Ryzen Threadripper PRO 9955WX
- Ubuntu 24.04 LTS

알고리즘에는 시간 제한이 있으며, 문제마다 다를 수 있습니다. 비공개 평가 문제의 시간 제한은 대회 종료 후에 공개됩니다. 일반적인 시간 제한은 몇 분에서 30분까지 다양할 것으로 예상되며, 실제 경과 시간(wall-clock time) 기준입니다. 팀들의 개발 환경과 서버 사양이 다를 수 있으므로, 알고리즘은 내부 경과 시간을 주의 깊게 모니터링하여 제한을 초과하지 않도록 해야 합니다.

공정한 경쟁을 위해 제출된 알고리즘에는 다음과 같은 제약이 적용됩니다:

- 실행 중 외부 인터넷 접근 불가
- 실행 폴더 상위 디렉터리 접근 불가
- 최대 4 개의 CPU 코어(CPU 사용률 400%)
- 최대 16GB 메모리

이러한 제약을 위반하면 실격될 수 있습니다. 인터넷 차단 및 CPU 코어 제한은 평가 시 firejail과 cputlimit을 사용하여 적용됩니다.

3.3 리더보드

리더보드 순위는 점수 기반입니다. 각 비공개 테스트 인스턴스에 대해 알고리즘이 생성한 솔루션을 기반으로 다음과 같은 점수가 부여됩니다:

- -1: 솔루션이 실현 불가능하거나, 시간 제한을 초과하거나, 충돌이 발생한 경우
- R - nb: 솔루션이 실현 가능한 경우. 여기서 R은 평가된 팀 수이고, nb는 이 문제에서 더 좋은 목적 함수 값을 달성한 팀의 수

리더보드 순위는 리더보드가 생성되는 시점에 모든 문제에 걸친 총 점수로 결정됩니다. 솔루션의 실현 가능성은 제공된 `utils.py` 모듈의 `check_feasibility()` 함수로 검사됩니다.

최종 수상자는 리더보드 순위뿐만 아니라 기술 보고서와 발표를 함께 고려하여 결정됩니다. 따라서 최종 판단은 리더보드 순위, 기술 보고서, 발표의 세 가지 기준을 바탕으로 합니다. 새로운 아이디어나 이론적 혁신은 완전한 알고리즘 성능으로 이어지지 않더라도 높이 평가 될 것입니다.

4. 알고리즘 개발 및 기준 알고리즘

4.1 개발 환경

각 팀은 제출 전에 자신의 컴퓨터에서 알고리즘을 구현하고 테스트해야 합니다. 제출된 알고리즘은 miniforge로 생성된 conda 환경에서 실행됩니다. 서버 conda 환경 파일은 대회 웹 페이지에서 다운로드할 수 있습니다.

conda 환경에서 사용 가능한 주요 Python 패키지와 버전:

```
- python 3.12
- pandas 2.2.3
- networkx 3.4.2
- scipy 1.15.2
- scikit-learn 1.6.1
- numba 0.61.0
- cython 3.0.10
- openjdk 17
- dotnet-runtime 8
- ortools 9.15.6755
- gurobipy 13.0.2
- xpress 9.8.1
- gymnasium 1.2.3
- torch 2.11.0
- tensorflow 2.21.0
- keras 3.14.0
- shapely 2.1.2
```

팀들은 필요한 파일들을 zip으로 압축하여 다른 Python 패키지나 라이브러리를 사용할 수 있습니다. 평가 서버에는 전용 GPU가 없으므로, ML 관련 알고리즘은 CPU에서 실행되어야

합니다.

Gurobi와 FICO의 지원으로 다음 상용 MIP 솔버를 사용할 수 있습니다:

- Gurobi 13.0.2
- Xpress 9.8.1

4.2 기준 알고리즘

팀들의 알고리즘 개발을 돕기 위해 대회 웹페이지에서 간단한 그리디 알고리즘을 제공합니다. myalgorithm.py는 알고리즘의 진입점(entry point)입니다. 평가 서버는 이 파일과 다음 함수를 찾습니다:

```
def algorithm(prob_info, timelimit=60):  
    # 여기서 다른 모듈을 임포트하거나 추가 함수를 정의할 수 있습니다.  
    import baseline_greedy  
    return baseline_greedy.greedyalgorithm(prob_info, timelimit)
```

algorithm(prob_info, timelimit=60) 함수는 주어진 문제 prob_info에 대한 솔루션 정보를 담은 Python 딕셔너리를 반환해야 합니다.

솔루션 딕셔너리의 형식:

```
{ "operations": {  
    "0": [ // 날짜 인덱스  
        { "type": "ENTRY", "block_id": 48, "bay_id": 1, "x": 0, "y": 0,  
"orient_idx": 0 },  
        { "type": "ENTRY", "block_id": 53, "bay_id": 0, "x": 0, "y": 0,  
"orient_idx": 0 }  
    ],  
    "28": [  
        { "type": "EXIT", "block_id": 25, "bay_id": 2 }, // EXIT 먼저  
        { "type": "ENTRY", "block_id": 15, "bay_id": 0, "x": 106, "y": 0,  
"orient_idx": 0 }  
    ]  
}
```

솔루션 딕셔너리는 "operations"라는 단일 키를 가지며, 그 값은 날짜별 작업 딕셔너리입니다. 작업이 있는 날짜만 명시하면 됩니다. 작업 수에는 제한이 없지만, EXIT 작업은 반드시 ENTRY 작업보다 먼저 수행되어야 합니다. 블록의 위치("x", "y")는 소수가 될 수 없으며, 항상

정수여야 합니다.

기준 알고리즘은 EDD(Earliest Due Date, 납기일 우선) 규칙에 기반한 그리디 알고리즘을 구현합니다. 납기일이 가장 빠른 다음 블록을 AABB(축 정렬 경계 상자)의 오른쪽 하단 꼭짓점 들인 '후보 지점' 중 하나에 배치합니다. 크레인 제약이 첫 번째 단계에서 무시되어 결과 솔루션이 실현 불가능할 수 있습니다. 기준 알고리즘은 실현 불가능한 블록을 베이가 비워진 이후 날짜로 이동시킵니다.

4.2.1 다른 프로그래밍 언어 사용

Python이 아닌 언어와 라이브러리도 사용할 수 있습니다. 필요한 모든 바이너리와 라이브러리를 zip에 포함하고 myalgorithm.py에서 호출해야 합니다. 일부 언어에 대한 일반 지침:

- C/C++: Linux/Ubuntu 24.04 에서 먼저 컴파일한 후, 생성된 .so 또는 실행 파일을 zip 에 제출하세요. 서버 측 컴파일은 지원되지 않습니다.
- Java: 평가 서버에 openjdk 17 이 사용 가능합니다.
- Dotnet: 평가 서버에 Dotnet 런타임 8.0 이 사용 가능합니다.

4.3 알고리즘 테스터 및 솔루션 시각화

알고리즘을 테스트하고 솔루션을 시각화하는 간단한 방법을 제공합니다. 대회 웹사이트에서 alg_tester.zip을 다운로드하여 압축 해제한 후, ogc2026 conda 환경을 활성화하고 python alg_tester_app.py를 실행하면 됩니다. 풀 문제 인스턴스를 선택하고 myalgorithm.py가 포함된 디렉토리를 지정합니다. "Problem" 탭에서 문제, 베이, 블록 세부 정보를 확인할 수 있으며, "Solution" 탭에서는 선택한 알고리즘의 솔루션을 단면도를 통해 베이 내 블록 배열로 시각화하여 보여줍니다.

5. 대회 형식 및 정책

OGC 2026은 예선 단계와 본선 단계, 두 단계로 진행됩니다. 참가자들은 수학적 최적화, 머신러닝, 휴리스틱, 하이브리드 접근법 등 어떤 방법론이든 자유롭게 활용하여 문제를 풀 수 있습니다.

리더보드 정책: 각 단계 전체에서 리더보드는 정확한 순위 대신 구간(예: 상위 10, 20, 30위 등)으로 순위를 표시합니다. 각 단계의 정확한 최종 순위는 해당 단계의 모든 평가가 완료된

후에만 공개됩니다.

5.1 예선 단계

5. **자동 평가:** 참가자들이 대회 서버에 알고리즘 코드를 제출하면 서버가 자동으로 코드를 실행하고 성능을 평가합니다. 리더보드는 최신 구간 순위를 반영하여 업데이트됩니다.
6. **본선 진출:** 자동 평가 결과와 조직위원회의 코드 검증을 바탕으로 상위 30~40 개 팀(변동 가능)이 본선 진출 대상으로 선발됩니다. 예선의 최종 순위는 해당 단계 종료 시 발표됩니다.

5.2 본선 단계

7. **자동 평가:** 예선과 마찬가지로 참가자들이 대회 서버에 알고리즘 코드를 제출하여 자동 실행 및 구간 점수를 받습니다.
8. **기술 보고서 제출:** 본선 진출 팀은 모두 지정된 형식으로 알고리즘을 설명하는 기술 보고서를 제출해야 합니다. 제출된 코드의 무결성과 품질, 기술 보고서는 조직위원회가 별도로 검토합니다.
9. **발표:** 본선 진출 팀은 조직위원회 앞에서 접근 방법과 결과를 발표해야 하며, 이는 최종 평가에 반영됩니다.
10. **최종 선발:** 모든 평가가 완료되면 조직위원회가 자동 점수, 기술 보고서, 코드 평가, 발표를 종합적으로 검토하여 최종 수상 팀을 결정합니다. 완전한 최종 순위는 이 시점에 발표됩니다.

5.3 평가, 공개 및 청렴성

구체적인 평가 기준과 절차는 조직위원회의 재량에 따라 결정 및 운영됩니다. 수상 팀의 최종 순위 외에 개인별 평가 결과는 공개되지 않습니다.

본선 진출 팀이 제출한 모든 알고리즘 코드와 기술 보고서는 공개됩니다. 따라서 제출된 모든 코드는 오픈소스로 공개될 수 있어야 합니다. 참가함으로써 팀들은 이러한 공개에 동의합니다.

최종 결과 발표 후에도 표절, 제한된 자원의 미공개 사용 또는 기타 대회 규정 위반을 포함한 부정행위가 발견되거나 신고되어 조직위원회의 검토를 통해 사실로 확인될 경우 수상이 취소될 수 있습니다.

6. 조직위원회

OGC 2026은 다음 분들에 의해 조직 및 운영됩니다:

- 이경식 (서울대학교, 위원장)
- 이충목 (한국외국어대학교)
- 우종훈 (서울대학교)
- 강찬목 (송실대학교)
- 정슬기 (아주대학교)
- 정윤우 (LG CNS)
- 안기백 (LG CNS)
- 이제리미 (LG CNS)